

WEB SCRAPING

EXTRACTING DATA FROM THE WEB

Turn any website into **structured data**

WHAT IS WEB SCRAPING?

Writing a program that **automatically reads websites** and pulls out the data you need — like a robot that browses the web for you.

DATA ANALYSIS

Collect prices, reviews, or stats from many pages at once

RESEARCH

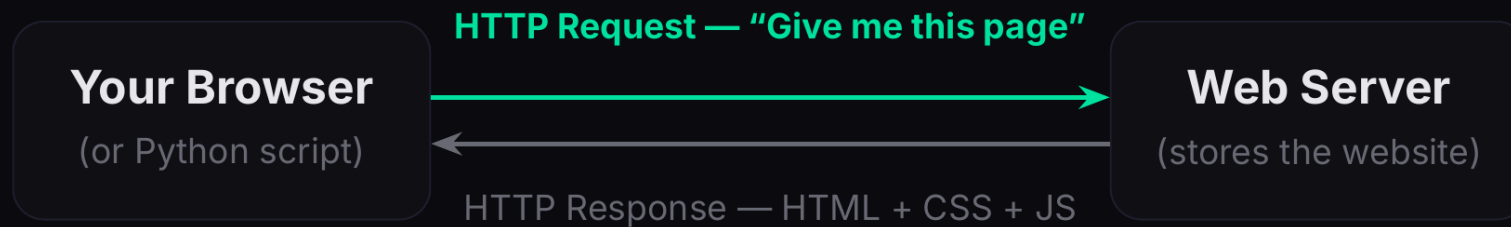
Gather public data for academic studies or reports

MONITORING

Track changes on websites over time (prices, availability)

HOW THE WEB WORKS

Your browser asks a server for a page. The server sends back
HTML.



200 OK

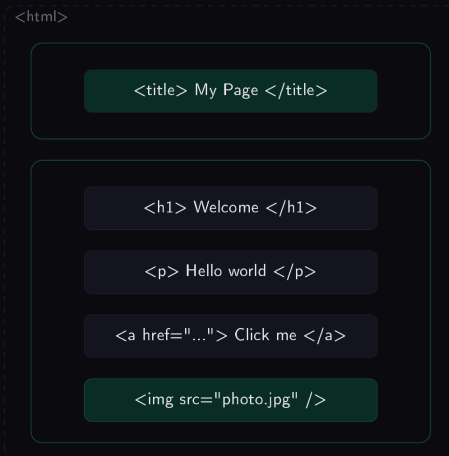
Page found, here it is

403 FORBIDDEN

You're not allowed in

HTML — THE LANGUAGE OF WEB PAGES

Every web page is made of **HTML tags** — like labeled boxes inside boxes.



EXAMPLE.HTML

```
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Welcome</h1>
    <p class="intro">Hello world</p>
    <a href="/about">Click me</a>
  </body>
</html>
```

PLAY BY THE RULES

Web scraping is powerful, but you need to be **responsible**.

- 1. Check robots.txt** — every site has a `/robots.txt` file that says what you may scrape
- 2. Add delays** — use `time.sleep()` between requests so you don't overload their server
- 3. Respect ToS** — if the Terms of Service say "no scraping," don't do it

Rule of thumb: if you wouldn't explain your scraping to the site owner,
don't do it.

// THINK

A company blocks all scrapers from their website. Is it ever okay to get around their block? When might it be justified, and when is it clearly wrong?

// LET'S CODE

TWO PYTHON LIBRARIES FOR EVERY SCRAPING JOB

DOWNLOAD A WEB PAGE

`requests` — the simplest way to fetch a page with Python

FETCH_PAGE.PY

```
1 import requests
2
3 # Fetch a web page
4 response = requests.get("http://quotes.toscrape.com")
5
6 print(response.status_code)
7 # 200
8
9 print(response.text[:80])
10 # <!DOCTYPE html><html lang="en"><head>...
```

One line to download any web page: `requests.get(url)`

PARSE THE HTML

BeautifulSoup — search through HTML and extract data

PARSE_QUOTES.PY

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 response = requests.get("http://quotes.toscrape.com")
5 soup = BeautifulSoup(response.text, "html.parser")
6 quotes = soup.find_all("div", class_="quote")
7
8 for q in quotes[:3]:
9     text = q.find("span", class_="text").get_text()
10    author = q.find("small", class_="author").get_text()
11    print(author, "-", text[:50])
12 # Albert Einstein - "The world as we have created ..."
```

requests downloads → **BeautifulSoup** parses → you extract what you need

INSPECT BEFORE YOU SCRAPE

Use your browser's **Developer Tools** to find the right HTML tags.

How to inspect:

1. Right-click on the data you want
2. Click **"Inspect"**
3. Look at the tag name and class
4. Use that in your `find()` call

Useful DevTools tabs:

Elements — see the HTML structure

Network — see all requests the page makes

Console — test selectors with
`$$("selector")`

Pro tip: Always inspect the page first. Never guess at tag names or classes.

HANDLE MULTIPLE PAGES

Most websites split data across pages — loop through them all.

PAGINATION.PY

```
1 import requests
2 from bs4 import BeautifulSoup
3 import time
4
5 all_titles = []
6 base = "http://books.toscrape.com/catalogue/page-{}.html"
7
8 for page in range(1, 4):
9     soup = BeautifulSoup(
10         requests.get(base.format(page)).text, "html.parser")
11     for b in soup.find_all("article", class_="product_pod"):
12         all_titles.append(b.find("h3").find("a")["title"])
13     time.sleep(1) # be polite!
```

Find the URL pattern, loop through pages, and always **add a delay**.

SAVE YOUR DATA

Don't just print results — save them to a **CSV** file.

SAVE_CSV.PY

```
1 import csv
2
3 data = [
4     {"title": "Book A", "price": "£20"},
5     {"title": "Book B", "price": "£15"},
6 ]
7
8 with open("books.csv", "w", newline="") as f:
9     writer = csv.DictWriter(
10         f, fieldnames=["title", "price"])
11     writer.writeheader()
12     writer.writerows(data)
13 # Creates books.csv that opens in Excel
```

csv is built into Python — no extra install needed. Output opens directly in Excel.

CONTROL A REAL BROWSER

Selenium — for JavaScript-heavy sites where requests won't work

SELENIUM_EXAMPLE.PY

```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3
4 driver = webdriver.Chrome()
5 driver.get("http://quotes.toscrape.com/js/")
6
7 quotes = driver.find_elements(By.CLASS_NAME, "quote")
8 for q in quotes[:3]:
9     text = q.find_element(By.CLASS_NAME, "text").text
10    author = q.find_element(By.CLASS_NAME, "author").text
11    print(author, "-", text[:50])
12
13 driver.quit()
```

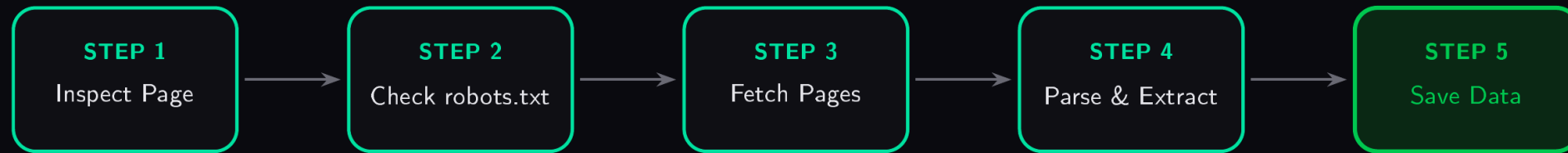
Use Selenium when content loads via **JavaScript**. Use requests + BS4 for everything else.

WHICH TOOL SHOULD I USE?

TOOL	SPEED	JAVASCRIPT?	DIFFICULTY	BEST FOR
requests + BS4	Very fast	No	Easy	Simple, static pages
Selenium	Slow	Yes	Medium	JS-heavy sites, forms

Start simple. Use **requests + BeautifulSoup** first. Only switch to Selenium when you actually need it.

THE 5-STEP SCRAPING PIPELINE



-
1. **Inspect** the page in DevTools — find the tags and classes you need
 2. **Check** robots.txt & ToS — make sure scraping is allowed
 3. **Fetch** the HTML with requests (or Selenium if JS is needed)
 4. **Parse** the HTML with BeautifulSoup and extract data

// EXERCISE

You want to scrape a new website you've never visited before. List 3 things you should check before writing any code.

KEY TAKEAWAYS

- > **Inspect first** — always use DevTools before writing code

- > **Be ethical** — check robots.txt, add delays, respect ToS

- > **Start simple** — requests + BeautifulSoup handles 90% of tasks

- > **Save as you go** — don't wait until the end to store data

GO FURTHER

DOCUMENTATION

- requests.readthedocs.io
- crummy.com/software/BeautifulSoup
- selenium-python.readthedocs.io

PRACTICE SITES

- quotes.toscrape.com
- books.toscrape.com
- httpbin.org

BOOKS

- Web Scraping with Python — Mitchell
- Automate the Boring Stuff — Sweigart

QUESTIONS?

Let's discuss what we've learned.